# Frontend Theory Assignment - 3

JavaScript | Deadline: 22 Oct 2026

## Q1. What is the difference between frontend and backend JavaScript?

**Frontend JavaScript** runs in the user's browser (Chrome, Firefox, etc.) and controls what the user sees and interacts with. **Backend JavaScript** runs on a server, usually using **Node.js**, and handles business logic, databases and APIs.

| Point | Frontend JS | Backend JS |
|---|---|---|
| Runs in | Browser | Server (Node.js) |
| Purpose | UI, interactivity, validation | Data processing, auth, APIs |
| Access to | DOM, window, localStorage | File system, database, network, OS |
| Examples | React, Vue, plain JS | Express, NestJS, Node.js |

## Q2. What is DOM?

The **DOM (Document Object Model)** is a tree-like, in-memory representation of an HTML page created by the browser. Every element, attribute and piece of text becomes a *node* that JavaScript can read, change, add or remove, so the page can be updated without reloading.

```
const heading = document.getElementById("title");
heading.textContent = "Hello DOM";
heading.style.color = "blue";
```

## Q3. What is event in JavaScript?

An **event** is an action or occurrence in the browser that JavaScript can respond to, such as a click, key press, mouse move, form submit or page load. We attach an **event listener** (handler function) to run code when the event occurs.

```
const btn = document.querySelector("#btn");
btn.addEventListener("click", function (e) {
  console.log("Button clicked!");
});
```

## Q4. What is event loop in JavaScript?

JavaScript is **single-threaded** (one call stack), yet it handles asynchronous work like timers, network requests and events. The **event loop** makes this possible. It continuously checks whether the **call stack** is empty and, if so, moves the next task from a queue onto the stack.

- **Call stack:** where functions currently execute.
- **Web APIs:** browser features (setTimeout, fetch, DOM events) that run async work outside the stack.
- **Microtask queue:** promise callbacks (.then, async/await); has higher priority.
- **Callback (macrotask) queue:** setTimeout, events, etc.

Microtasks run before macrotasks, so the output below is Start, End, Promise, Timeout:

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
```

```
console.log("End");
```

## Q5. What is hoisting in JavaScript?

**Hoisting** is JavaScript's behaviour of moving declarations to the top of their scope during the creation phase, before the code runs.

- **var:** hoisted and initialised with *undefined*.
- **let / const:** hoisted but not initialised (Temporal Dead Zone); using them early throws a ReferenceError.
- **Function declarations:** fully hoisted, so they can be called before they are defined.

```
console.log(a);   // undefined
var a = 5;

sayHi();          // works
function sayHi() { console.log("Hi"); }

console.log(b);   // ReferenceError
let b = 10;
```

## Q6. What is event bubbling?

**Event bubbling** is the process where an event triggered on an element first runs its own handler and then propagates (bubbles) upward through its ancestors: parent, grandparent, up to the document. The opposite is **capturing**, which goes from the top down. Bubbling can be stopped with *event.stopPropagation()* and enables **event delegation** (one listener on a parent handling many children).

```
<div id="parent">
  <button id="child">Click</button>
</div>

parent.addEventListener("click", () => console.log("Parent"));
child.addEventListener("click", () => console.log("Child"));
// Clicking the button prints: Child, then Parent
```

## Q7. What is the difference between named function, anonymous function and arrow function?

- **Named function:** has a name; hoisted (if a declaration); name helps in debugging and recursion.
- **Anonymous function:** has no name; usually passed as a value or callback.
- **Arrow function (ES6):** shorter syntax using =>; no own *this* (uses the surrounding this), no *arguments* object, cannot be used as a constructor.

```
// Named
function add(a, b) { return a + b; }

// Anonymous
const sub = function (a, b) { return a - b; };
setTimeout(function () { console.log("done"); }, 1000);

// Arrow
const mul = (a, b) => a * b;
```

## Q8. Explain for-of and for-in loop?

- **for...in** loops over the enumerable **keys (property names)** of an object. For arrays it gives indexes (as strings). Best for objects.
- **for...of** loops over the **values** of iterable objects such as arrays, strings, Maps and Sets. Not for plain objects.

```
const arr = ["a", "b", "c"];
for (let i in arr) console.log(i);      // 0, 1, 2
for (let v of arr) console.log(v);      // a, b, c

const user = { name: "Ravi", age: 20 };
for (let k in user) console.log(k, user[k]);  // name Ravi, age 20
```

## Q9. Explain any five array methods?

- **push() / pop():** push adds an element at the end; pop removes the last element.
- **map():** creates a new array by applying a function to every element.
- **filter():** creates a new array with only the elements that pass a condition.
- **reduce():** reduces the array to a single value (sum, product, etc.).
- **forEach():** runs a function for each element (returns nothing).

```
const n = [1, 2, 3, 4, 5];

n.push(6);                          // [1,2,3,4,5,6]
n.pop();                            // removes 6
n.map(x => x * 2);                  // [2,4,6,8,10]
n.filter(x => x % 2 === 0);         // [2,4]
n.reduce((sum, x) => sum + x, 0);   // 15
n.forEach(x => console.log(x));     // prints each value
```

## Q10. What is callback method?

A **callback** is a function passed as an argument to another function, to be executed later, usually after a task finishes. Callbacks are the basis of asynchronous JavaScript (timers, events, file/network requests). Too many nested callbacks lead to "callback hell", which Promises and async/await help solve.

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}
greet("Asha", function () {
  console.log("Callback executed");
});

setTimeout(() => console.log("After 1 second"), 1000);
```